

Cours technique : analyse et reconstruction vasculaire avec `medical_build`

Objectifs

Ce cours présente une chaîne Python reproductible pour analyser une géométrie vasculaire complexe, extraire un réseau de centerlines, construire des sections orientées, sérialiser les données et préparer une reconstruction CAD ou CFD. L'architecture sépare volontairement l'analyse géométrique de la reconstruction.

1. Représentation d'une surface vasculaire

Une surface triangulée est un ensemble de sommets $V = \{x_i\}$ et de cellules triangulaires T . La surface doit être orientable et, pour définir un volume, fermée. Les boucles de bord sont les arêtes utilisées par une seule cellule. Une arête utilisée par plus de deux cellules indique une non-manifoldité.

Pour une surface paramétrique locale $S(u, v)$, la normale est

$$n = (S_u \times S_v) / ||S_u \times S_v||.$$

Les contrôles fondamentaux sont donc la finitude des coordonnées, l'absence de cellules dégénérées, la cohérence des normales, la fermeture des arêtes et la connectivité.

2. Delaunay et Voronoi

La triangulation de Delaunay maximise le minimum des angles dans le cas bidimensionnel. En trois dimensions, elle décompose un nuage en tétraèdres dont les sphères circonscrites ne contiennent aucun autre point. Le diagramme de Voronoi est dual de Delaunay : chaque cellule contient les points de l'espace plus proches d'un site donné que des autres sites.

Dans la reproduction VMTK, il faut distinguer le Voronoi volumique construit dans les tétraèdres internes du Voronoi limité aux sommets de surface. Les pôles associés aux sections doivent être reliés à la topologie interne, sinon les chemins obtenus peuvent être artificiellement courts.

3. Centerline et coût géométrique

Une centerline est un chemin dans le lumen qui relie une entrée à une sortie. Pour un graphe $G=(N, E)$, Dijkstra minimise

$$d(v) = \min_{\{p:s \rightarrow v\}} \sum_{\{e \in p\}} w(e).$$

Le poids peut combiner la longueur et un terme de rayon :

$$w(e) = \alpha ||x_i - x_j|| + \beta / \max(r_e, \epsilon).$$

La conservation des prédécesseurs est indispensable pour reconstruire le chemin inverse depuis la cible. Le résultat doit être contrôlé par sa longueur, sa tortuosité, sa distance à la paroi et sa continuité tangentielle.

4. Repères et sections

À chaque station $x(s)$, on associe une tangente t , une normale n et une binormale $b=t \times n$. Un repère orthonormal vérifie

$$||t||=||n||=||b||=1, t \cdot n=t \cdot b=n \cdot b=0.$$

Une section est l'intersection du lumen avec un plan local

$$\Pi(s) = \{x : (x-x(s)) \cdot t(s)=0\}.$$

Le contour doit être ordonné, fermé et phase-locké entre stations. Cette dernière propriété évite les rotations arbitraires de profil qui créent des torsions artificielles dans les lofts.

5. Contrat sérialisable

SectionRecord contient le centre, la tangente, le repère local, les points bruts, les points phase-lockés, l'aire, le périmètre et le rayon équivalent. BranchRecord contient les points centerline, les abscisses, les tangentes, les caps source/cible et les sections. GeometryAnalysisData contient l'ensemble du réseau et les diagnostics.

Le contrat est validé avant écriture JSON. Les tableaux numériques peuvent être exportés en NPZ compressé pour conserver les dimensions et les types sans sérialiser des objets Python.

6. Reconstruction Build123d

Les profils OCC doivent être nettoyés : suppression des doublons consécutifs, suppression du dernier point s'il répète le premier et projection sur le plan tangent. Le loft lisse minimise généralement le nombre de faces ; le loft réglé suit plus directement les profils mais peut coûter davantage.

Une branche doit être validée seule avant fusion. Une union globale exige des volumes correctement orientés, un recouvrement réel au carrefour et une validité OCC après chaque opération booléenne. Un Compound de solides séparés n'est pas équivalent à un volume fluide unique.

7. BlockMesh global

Un maillage blockMesh est une topologie de blocs hexaédriques. Le registre global doit partager les mêmes indices de sommets et les mêmes faces sur les interfaces internes. Une génération indépendante de huit branches donne huit composantes et ne constitue pas un domaine global.

GlobalBlockMesh vérifie les blocs dégénérés, l'usage des faces, les faces non-manifold et la connexité. Le carrefour anatomique doit être modélisé par un noyau multi-blocs dont les ports correspondent aux sections réelles.

8. STL et CFD

Une surface STL CFD doit être contrôlée avec plusieurs niveaux : arêtes frontières, arêtes non-manifold, doublons, triangles dégénérés, normales, fermeture et volume signé. Les patches inlet, outlet_* et wall doivent être identifiables dans constant/triSurface et vérifiés après snappyHexMesh dans constant/polyMesh/boundary.

La séquence de validation est :

surface source → caps exacts → STL patches → blockMesh → snappyHexMesh → checkMesh

La présence d'un fichier STL ne prouve pas son absence de trous. Les surfaces produites par une classification heuristique doivent rester marquées comme provisoires tant que sur faceCheck et checkMesh n'ont pas confirmé

la topologie.

9. Reproductibilité et benchmarks

Le cas complexe contient les centerlines, les diagnostics, les rapports Build123d et Classy Blocks, les STL et les dictionnaires OpenFOAM. Les benchmarks mesurent le temps, la validité OCC, le volume signé, le nombre de faces et la connectivité. Toute modification future doit comparer ces métriques à une baseline.

Références

[1]: <https://vtk.org/> "VTK documentation"

[2]: <https://build123d.readthedocs.io/> "Build123d documentation"

[3]: https://doc.cgal.org/latest/Triangulation_3/index.html "CGAL 3D triangulations"

[4]: https://github.com/damogranlabs/classy_blocks "Classy Blocks"

[5]: <https://www.openfoam.com/documentation/guides/latest/doc/guide-meshing-snappyhexmesh.html> "OpenFOAM snappyHexMesh"